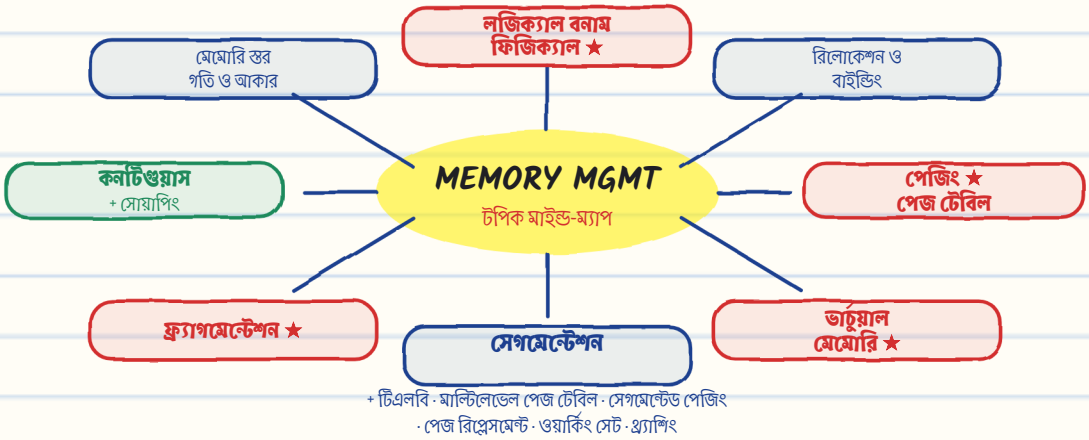
মেমোরি ম্যানেজমেন্ট

পেজিং · সেগমেন্টেশন · টিএলবি · পেজ ফল্ট · রিপ্লেসমেন্ট · ১১ পাতায় পরীক্ষার পূর্ণ প্রস্তুতি



↑ পুরো টপিক এক পাতায়: লাল = বেশি আসে, সবুজ = যে পদ্ধতি থেকে প্রথম প্রশ্নগুলো আসে

➜ সিলেবাসের যে অংশ

সিলেবাসে আছে "*Memory management: swapping, paging, segmentation, virtual memory*"। শিডিউলিং ও IPC প্রসেস লেসনে আছে; ডিস্কের গঠন ইনপুট/আউটপুট লেসনে।

হাই-ইল্ড তালিকা

① পেজিং বাহ্যিক ফ্র্যাগ ঠিকায়, ভেতরেরটা নয়
② $p = LA \text{ div } P$ · $d = LA \bmod P$
③ $PA = f \cdot P + d$ · অফসেট = $\log_2 P$ বিট
④ টিএলবি হিট = ১ অ্যাক্সেস, মিস = ২
⑤ 32-bit + 4 KB ⇒ 2^{20} এন্ট্রি = 4 MB
⑥ সেগ: $PA = Base[s] + d,\ d < Limit[s]$
⑦ ভিন্ন দৈর্ঘ্য সেগমেন্ট ⇒ বাহ্যিক ফ্র্যাগ
⑧ ফল্ট: ট্র্যাপ, টেস্ট, নাও, পড়ো, PTE, আবার
⑨ $EAT = (1 - p)m + p \cdot f$
⑩ থ্র্যাশিং ⇒ কম প্রসেস রাখো

সংকেত হলুদ = মূল শব্দ · লাল বৃত্ত = অবশ্যই মুখস্থ · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

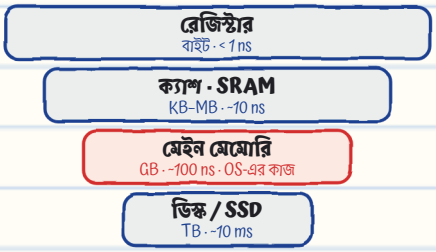
➜ সূচিপত্র

✓ মেমোরি ম্যানেজমেন্ট কেন ... পৃ. ২
✓ ফ্র্যাগমেন্টেশন ও সোয়াপিং ... পৃ. ৩
✓ পেজিং ও অনুবাদ ... পৃ. ৪
✓ টিএলবি, মাল্টিলেভেল, PTE বিট ... পৃ. ৫
✓ সেগমেন্টেশন ... পৃ. ৬
✓ ভার্চুয়াল মেমোরি ও পেজ ফল্ট ... পৃ. ৭
✓ পেজ রিপ্লেসমেন্ট ... পৃ. ৮
✓ ওয়ার্কিং সেট ও থ্র্যাশিং ... পৃ. ৯
✓ রাতের পাতা পৃ. ১০ · অনুশীলন পৃ. ১১

এই পুরো লেসনের উদ্দেশ্য মেমোরিই একমাত্র রিসোর্স যা ছাড়া কোনো প্রসেস চলতে পারে না, আবার সবচেয়ে সহজেই অতিরিক্ত বরাদ্দ হয়ে যায় এমন রিসোর্স। এখানকার প্রতিটি ধারণা — স্থির-আকারের পেজ, সরানো নয় বরং হুবহু নকল করা অফসেট, ডিস্কে লেখা হবে কি না তা ঠিক করা ডার্টি বিট — একসঙ্গে তিনটি প্রতিশ্রুতি রাখে: **প্রোগ্রাম পায় সমতল অ্যাড্রেস স্পেস**, **দুই প্রসেস কখনো ওভারল্যাপ করে না**, আর **OS যেকোনো সময় যেকোনো কিছু সরাতে পারে**।

১ স্তরবিন্যাস, বাইন্ডিং ও অ্যাড্রেস মূল বিষয় ★

সংজ্ঞা **মেমোরি ম্যানেজমেন্ট** *(Memory management)* নিয়ন্ত্রণ করে মেমোরি কীভাবে প্রসেসে বরাদ্দ ও ফেরত নেওয়া হয়। এর দুটি লক্ষ্য: **মেমোরির ব্যবহার সর্বাধিক করা** এবং **সোয়াপের সময় সর্বনিম্ন করা**। এটি আরও তিনটি জিনিস দেয় — **অ্যাবস্ট্রাকশন** (মেমোরি একটি সমতল অ্যাড্রেস তালিকা বলে মনে হয়), **প্রোটেকশন** (এক প্রসেস অন্যের মেমোরি ছুঁতে পারে না) এবং **শেয়ারিং**।



উপরে গেলে দ্রুত ও ছোট। OS কেবল **মাঝের স্তরটি** সামলায়

স্তর	আকার	অ্যাক্সেস	কে সামলায়
রেজিস্টার	বাইট	< 1 ns	কম্পাইলার
ক্যাশ *(SRAM)*	KB – MB	1 – 100 ns	হার্ডওয়্যার
মেইন মেমোরি *(DRAM)*	GB	50 – 100 ns	**OS — এই লেসন**
সেকেন্ডারি স্টোরেজ	TB	50 µs – 10 ms	ফাইল সিস্টেম + সোয়াপ

বাইন্ডিং সময়	অ্যাড্রেস কখন ঠিক হয়	প্রসেস সরলে কী হয়
কম্পাইল টাইম	কম্পাইলের সময়ই; কাঠামো তখনই জানা — পরম শুধু অ্যাড্রেস	**আবার কম্পাইল করতে হবে**
লোড টাইম	মেমোরিতে ঢোকানোর সময় — রিলোকেটেবল কোড, লোডের সময়ই বেস + লিমিট বসে	শুধু আবার লোড করলেই হবে
এক্সিকিউশন টাইম ★	**প্রতিটি রেফারেন্সের** সময় — *MMU* অনুবাদ করে, বেস/লিমিট বা পেজ টেবিল দিয়ে	**কিছুই নয় — সত্যিকারের ডাইনামিক রিলোকেশন**

লজিক্যাল বনাম ফিজিক্যাল **লজিক্যাল (রিলেটিভ)** অ্যাড্রেসটি প্রোগ্রাম নিজেই তৈরি করে — *0* থেকে গোনা, প্রসেস কোথায় বসে তার সাথে কোনো সম্পর্ক নেই। **ফিজিক্যাল (অ্যাবসলিউট)** অ্যাড্রেস আসল *RAM* ইউনিটের ঠিকানা; **শুধু মেমোরি হার্ডওয়্যারই তা দেখে**। একটিকে অন্যটিতে বদলানোই **রিলোকেশন**, আর যেহেতু প্রোগ্রাম চলতে চলতেই এটি হয়, তাই *OS* যেকোনো মুহূর্তে প্রসেস সরাতে পারে।

বেস + লিমিট **রিলোকেশন (বেস) রেজিস্টার** = পার্টিশন যেখানে শুরু হয়; **লিমিট রেজিস্টার** = তার আকার। প্রতিটি লজিক্যাল অ্যাড্রেস *a* আগে লিমিটের সাথে মিলিয়ে দেখা হয়, তারপর ফিজিক্যাল = লজিক্যাল + বেস। যদি *a* ≥ লিমিট হয়, *CPU* ট্র্যাপ করে প্রসেস বন্ধ করে দেয়।

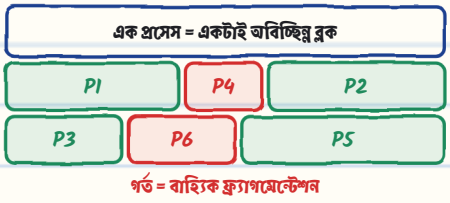
মনে রাখার কৌশল **"কম্পাইল = জমে গেল, লোড = বাঁধা, রান = চলমান।"** কেবল **এক্সিকিউশন টাইম** বাইন্ডিংয়েই প্রোগ্রাম সরানো যায় আবার কম্পাইল ছাড়াই — এজন্যই প্রতিটি আসল মেশিনে *MMU* আছে। আর এটা ধরে রাখো: প্রোগ্রাম লজিক্যাল দেখে, শুধু চিপ ফিজিক্যাল।

পরীক্ষার ফাঁদ *(i)* **লোড** টাইমে বাইন্ডিংয়েও রিলোকেটেবল কোড লাগে; সোয়াপ ও কম্প্যাকশনকে সম্ভব করে **এক্সিকিউশন** টাইম। *(ii)* পরম অ্যাড্রেস থাকলে প্রতিটি নতুন অবস্থানে **আবার কম্পাইল** করতে হয়। *(iii)* বেস + লিমিট **কনটিগুয়াস** মেমোরির, পেজ টেবিল **পেজিং**-এর।

২ ফ্র্যাগমেন্টেশন, কনটিগুয়াস বরাদ্দ ও সোয়াপিং

পেজিংয়ের আগে ★

সংজ্ঞা **কনটিগুয়াস অ্যালোকেশন** *(Contiguous allocation)* প্রতিটি প্রসেসকে মেমোরির **একটি অবিচ্ছিন্ন ব্লক** দেয়, যার জন্য OS একটি **বেস (রিলোকেশন)** রেজিস্টার ও একটি **লিমিট** রেজিস্টার বসায়। **ফ্র্যাগমেন্টেশন** *(Fragmentation)* হলো এমন ফাঁকা মেমোরি যা ব্যবহার করা যায় না: অভ্যন্তরীণ = বরাদ্দ দেওয়া ব্লকের **ভেতরের** ফাঁকা, বাহ্যিক = ব্লকের **মাঝের** ফাঁকা, পরের অনুরোধের জন্য যা কনটিগুয়াস নয়। ভেতরের অপচয় কখনো ফেরত পাওয়া যায় না; বাইরের অপচয় কেবল **কম্প্যাকশন** দিয়ে যায়।



শুধু সবচেয়ে বড় কনটিগুয়াস ফাঁকাই গোনা হয়, ফাঁকা মেমোরির যোগফল কখনো নয়

কম্প্যাকশন গর্ত বন্ধ করে; ভেতরের অপচয় কখনো নয়

কৌশল	যে গর্তটি বাছে	মূল্যায়ন
ফার্স্ট ফিট	শুরু থেকে স্ক্যান করে প্রথম যথেষ্ট বড় গর্ত	দ্রুত; **শেষে** সরু টুকরো রেখে যায়
বেস্ট ফিট	যে ছোট গর্তে মোড়ে	**সবচেয়ে খারাপ** অব্যবহারীয় টুকরো
ওয়ার্স্ট ফিট	সবচেয়ে বড় গর্তটি	বড় ব্লকটাই তাড়াতাড়ি নষ্ট করে
নেক্সট ফিট	যেখানে ছিল সেখান থেকেই শুরু করে প্রথম মিলে যায়	ক্ষতি সমানভাবে ছড়ায়

পরীক্ষার ফাঁদ *(i)* "**কোনটিতে বাহ্যিক ফ্র্যাগমেন্টেশন?**" → কনটিগুয়াস ও সেগমেন্টেশন, **কখনোই পেজিং নয়**; "অভ্যন্তরীণ কোথায়?" → পেজিং ও **ফিক্সড** পার্টিশন। *(ii)* কম্প্যাকশনের জন্য রিলোকেশন হার্ডওয়্যার লাগে, অর্থাৎ **এক্সিকিউশন টাইম** বাইন্ডিং। *(iii)* সোয়াপ করলে কাগজে ফাঁকা জায়গা দ্বিগুণও হতে পারে, তবু পরের প্রসেসের জন্য বড় ব্লক না থাকতে পারে।

সোয়াপিং (Swapping) **সোয়াপিং** একটি **পুরো প্রসেস** ডিস্কের সোয়াপ ডিভাইসে সরিয়ে দেয় এবং পরে আবার মেমোরিতে আনে, ফলে সিস্টেম ফিজিক্যাল মেমোরির চেয়ে বেশি প্রসেস চালাতে পারে, আর সুযোগ এলে কম্প্যাকশনও করা যায়। এর পুরো খরচ হলো ডিস্ক I/O। সোয়াপিং ≠ পেজিং: সোয়াপিং **পুরো প্রসেস** OS-এর নিজস্ব সিদ্ধান্তে সরায়; পেজিং **একটি পেজ** সরায় যখন প্রোগ্রাম নিজেই সেটি রেফারেন্স করে, আর প্রসেসের নজরে এটি ঘটে না।

সমাধানকৃত উদাহরণ ✍ একটি সোয়াপে কত খরচ?

ডিস্ক 10 MB/s গতিতে চলে, গড় seek ~10 ms। (ক) 1 MB প্রসেস বাইরে ও ভেতরে = 200 ms + 20 ms = 220 ms। (খ) 33 MB প্রসেস দুইবার করে বাইরে-ভেতরে = 132 MB ÷ 10 = 13.2 s। (গ) বদলে একটি 4 KB পেজ = 0.4 ms — ৫৫০ গুণ সস্তা। (ঘ) তাই শুধু দরকারি পেজগুলোই সরানো **পেজিং**ই মেশিনকে সাড়া দেয়।

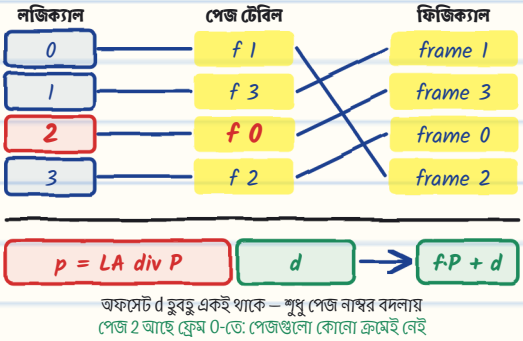
মনে রাখার কৌশল "**ভেতরে = অভ্যন্তরীণ, বাইরে = বাহ্যিক।**" কম্প্যাকশন কেবল বাইরেরটাই সারায় — ভেতরেরটা আগে দেওয়া আকারের ভেতরেই বেঁচে থাকে। কনটিগুয়াস বরাদ্দের জন্য একটা সম্পূর্ণ ফাঁকা গর্ত লাগে; পেজিংয় কখনো লাগে না — এজন্যই পেজিং বানানো হয়েছে।

বাস্তবে ডিমান্ড পেজিং প্রায় ক্লাসিক সোয়াপিংকে শেষ করে দিয়েছে। যথেষ্ট RAM থাকা মেশিন কখনোই সোয়াপ করে না; তবু হলে কার্নেল পুরো প্রসেস নয়, **ঠান্ডা অ্যানোনিমাস মেমোরি** পেজ করে বের করে, আর সোয়াপ ফাইল কাজের জায়গা নয়, শেষ অবলম্বন।

৩ পেজিং: পেজ, ফ্রেম, পেজ টেবিল, অনুবাদ

সবচেয়ে বেশি আসে ★

সংজ্ঞা **পেজিং** *(Paging)* দুই মেমোরিই স্থির-আকারের ব্লকে ভাগ করে: প্রসেসের লজিক্যাল মেমোরি **পেজে**, ফিজিক্যাল মেমোরি **ফ্রেমে**, **পেজ = ফ্রেম** আকারের। তাই প্রসেস যেকোনো ফ্রেমে যেকোনো ক্রমে বসতে পারে — কনটিগুয়াসতা আর থাকে না। **পেজ টেবিল** হলো পেজ টেবিল এন্ট্রি *(PTE)*-এর একটি অ্যারে, প্রতি পেজের জন্য একটি করে, যাতে ফ্রেম নম্বর ও নিয়ন্ত্রণ বিট থাকে।



প্রতি পেজের জন্য একটি ফ্রেম, যেকোনো ক্রমে — পেজিংয়ের পুরো ধারণা এই এক বাক্যেই আছে

পরিভাষা	অর্থ
পেজ	একটি প্রসেসের **লজিক্যাল** মেমোরির স্থির-আকারের ব্লক
ফ্রেম	**ফিজিক্যাল** মেমোরির স্থির-আকারের ব্লক, পেজের সমান আকারের
পেজ টেবিল	প্রতি প্রসেসের নিজস্ব PTE অ্যারে; এটিই প্রসেসের মানচিত্র
PTE	ফ্রেম নম্বর + ভ্যালিড / প্রোটেকশন / ডার্টি / রেফারেন্স
ফ্রি-ফ্রেম তালিকা	যে ফ্রেম কেউ ব্যবহার করছে না, সঙ্গে সঙ্গে ফাঁকা

পরীক্ষার ফাঁদ *(i)* অফসেট **হুবহু নকল** হয় — $f \cdot P$ ছাড়া আর কোথাও যোগ করা যায় না। *(ii)* **পেজিং কেবল বাহ্যিক ফ্র্যাগমেন্টেশন ঠিকায়**; শেষ পেজটা সাধারণত আংশিক খালি থাকে, প্রতি প্রসেসে গড়ে **অর্ধেক পেজ** নষ্ট হয়। *(iii)* বরাদ্দের একক ফ্রেম, তাই ফ্রেমের ভেতরের কিছু শেয়ার করা যায় না।

চারটি সূত্র — পেজিংয়ের প্রতিটি প্রশ্ন এর একটিই

পেজ নাম্বর $p = \lfloor LA \div P \rfloor = LA \text{ div } P$ **অফসেট** $d = LA \bmod P$

ফিজিক্যাল অ্যাড্রেস $PA = f \cdot P + d$ **প্রসেসের পেজ সংখ্যা** $size \div P$

যদি $P = 2^n$ বাইট হয় তবে অফসেটে ঠিক **$n = \log_2 P$ বিট** লাগে, বাকি বিটগুলো পেজ নম্বর নেয়। $4\ KB = 2^{12}$, তাই *32-bit* অ্যাড্রেস ভাগ হয় **20-বিট p + 12-বিট d**-এ।

সমাধানকৃত উদাহরণ ✍ লজিক্যাল 3000, পেজ 2 → ফ্রেম 5 $P = 1\ KB = 1024$। (ক) $p = 3000 \text{ div } 1024 = 2$ ($2 \times 1024 = 2048$; $3 \times 1024 = 3072 > 3000$)। (খ) $d = 3000 \bmod 1024 = 3000 - 2048 = 952$। (গ) $PTE[2] = 5$। (ঘ) $PA = 5 \times 1024 + 952 = 5120 + 952 =$ **6072**। (ঙ) যাচাই: $5 \times 1024 = 5120 \le 6072 < 6144$ ✓। (চ) অফসেট লজিক্যাল অ্যাড্রেস থেকে ফিজিক্যাল অ্যাড্রেসে সম্পূর্ণ অপরিবর্তিত যায়।

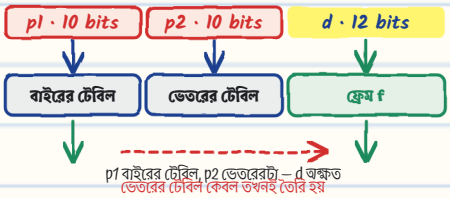
মনে রাখার কৌশল **"পেজ ঢোকে, ফ্রেম বেরোয়।"** পেজ নম্বর পেজ টেবিলে একটি **সূচক**, অফসেট ব্লকের ভেতরে একটি **অবস্থান**। পেজিং সূচক বদলায়, অবস্থান হুবহু রাখে।

IT link প্রতিটি *x86-64* ও *ARM64* CPU ঠিক এই টেবিলই হাঁটে, আর সাম্প্রতিক PTE একটি **টিএলবি**-তে রাখে — পরের পাতা।

৪ টিএলবি, মাল্টিলেভেল পেজ টেবিল ও PTE বিট

বেশি আসে ★

সংজ্ঞা প্রতিটি রেফারেন্সে পেজ টেবিল দেখলে প্রতি মেমোরি অ্যাক্সেসে **দুইবার** যেতে হতো। **ট্রান্সলেশন লুকঅ্যাসাইড বাফার (TLB)** হলো MMU-র ভেতরে থাকা একটি ছোট, খুব দ্রুত সাম্প্রতিক PTE-এর অ্যাসোসিয়েটিভ ক্যাশ। **হিট** হলে ফ্রেম নম্বর TLB থেকেই আসে আর ডেটা মাত্র **এক** অ্যাক্সেসে আসে; **মিস** হলে মেমোরি থেকে পেজ টেবিল পড়ে TLB নতুন করে ভরতে হয়।



4 KB পেজসহ 32-bit অ্যাড্রেস: ২০ বিট পেজ নম্বর, ভাগ 10 + 10

টিএলবি-র খরচ

ধরি t = TLB লুকআপ, m = একবার মেমোরি অ্যাক্সেস, h = হিট অনুপাত।

$$EAT = h\cdot(t + m) + (1 - h)\cdot(t + 2m)$$

পদ্ধতি, 32-bit + 4 KB পেজ	আকার
এক স্তরের	2^{20} এন্ট্রি × 4 B = প্রতি প্রসেসে 4 MB
দুই স্তরের (10 + 10)	বাইরে 2^{10} × 4 B = **4 KB**, সঙ্গে ব্যবহৃত প্রতি ভেতরের টেবিলে 4 KB
যা বাঁচে	৩২ পেজ ব্যবহারকারী প্রসেসের জন্য **4 KB + 128 B**, 4 MB নয়

সমাধানকৃত উদাহরণ ✍ টিএলবির সময় গণনা একটি TLB লুকআপে **20 ns**, একবার মেমোরি অ্যাক্সেসে **100 ns**, হিট অনুপাত **80 %**। (ক) হিট = 20 + 100 = **120 ns**। (খ) মিস = 20 + 100 (পেজ টেবিল) + 100 (ডেটা) = **220 ns**। (গ) EAT = 0.8 × 120 + 0.2 × 220 = 96 + 44 = **140 ns**। (ঘ) TLB ছাকরা হলে 200 ns। (ঙ) মাল্টিলেভেল টেবিল এই হিসাব বদলায় না — শুধু মিস ছোট হয়, কারণ বাইরের এন্ট্রি না থাকলেই হাঁটা ওখানেই শেষ।

PTE বিট	1 মানে	0 মানে	পড়ে কে
ভ্যালিড	পেজটা এখন কোনো ফ্রেমে আছে	মেমোরিতে নেই → **পেজ ফল্ট তোলো**	MMU, প্রতি রেফারেন্সে
প্রোটেকশন	পড়া / লেখা / চালানোর অনুমতি আছে	না → প্রোটেকশন ফল্ট	MMU, প্রতি রেফারেন্সে
ডার্টি	লোড হওয়ার পর লেখা হয়েছে	অপরিবর্তিত → ফিরিয়ে লিখব না	রিপ্লেসমেন্টের সময়
রেফারেন্স	সম্প্রতি ব্যবহৃত	ব্যবহৃত হয়নি	**ক্লক** ও LRU

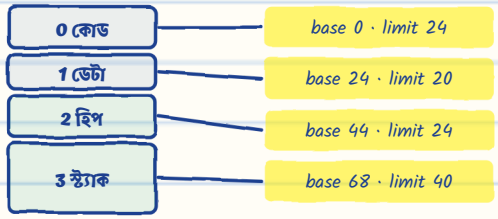
পরীক্ষার ফাঁদ (i) TLB **হার্ডওয়্যার**, মেমোরিতে থাকা টেবিল নয় — সাধারণত এটি **অ্যাসোসিয়েটিভ মেমোরি**, কনটেন্ট দিয়ে খোঁজা হয়। (ii) ডার্টি মানে "বদলেছে", রেফারেন্স মানে "দেখা হয়েছে"; দুটি শব্দ ভুল করলে উত্তর ভুল হয়। (iii) পেজ **ভ্যালিড থাকা সত্ত্বেও কখনো লেখা হয়নি** — ভ্যালিড মানে উপস্থিতি, পরিবর্তন নয়।

মনে রাখার কৌশল "**হিট একবার, মিস দুইবার।**" আর "**D = Dirty, R = Referenced** — ডার্টি ঠিক করে ফিরিয়ে লিখব কি না, রেফারেন্স ঠিক করে পরের কে বেরোবে।" TLB হলো পেজ টেবিলের ক্যাশ, তাই পেজগুলোর কাজের অংশ ছোট থাকলে টেবিল বড় হলেও সস্তা।

৫ সেগমেন্টেশন, এবং পেজিংসহ সেগমেন্টেশন

লিখিত প্রশ্নের প্রিয় ★

সংজ্ঞা **সেগমেন্টেশন** *(Segmentation)* মেমোরিকে সমান ব্লকের বদলে ভিন্ন দৈর্ঘ্যের লজিক্যাল একক — কোড, ডেটা, স্ট্যাক, হিপ — ভাগ করে। একটি লজিক্যাল অ্যাড্রেস হলো জোড়া **(সেগমেন্ট নম্বর, অফসেট)**। **সেগমেন্ট টেবিল** প্রতি সেগমেন্টে একটি **বেস** (শুরুর ফিজিক্যাল অ্যাড্রেস) ও একটি **লিমিট** (দৈর্ঘ্য) রাখে, সঙ্গে প্রোটেকশন বিট, তাই `d < Limit[s]` হলে `PA = Base[s] + d`; নয়তে **সেগমেন্টেশন ফল্ট**।



আবার সেই ভিন্ন-দৈর্ঘ্য পার্টিশনের সমস্যা। সমাধানকৃত: সেগমেন্ট 2-এর বেস 44, লিমিট 24, তাই (2, 23) → 67 ✓ কিন্তু (2, 24) → ফল্ট, কারণ 24 ≥ লিমিট

বিষয়	পেজিং	সেগমেন্টেশন
ব্লকের আকার	স্থির	**ভিন্ন**
অ্যাড্রেস	(p, d)	(s, d)
টেবিল এন্ট্রি	ফ্রেম নম্বর + বিট	বেস + লিমিট + বিট
ফ্র্যাগমেন্টেশন	শুধু অভ্যন্তরীণ	শুধু **বাহ্যিক**
ডেটার সাথে বাড়ে?	না	হ্যাঁ — হিপ, স্ট্যাক
প্রোটেকশন / শেয়ারিং	প্রতি পেজে, জটিল	প্রতি সেগমেন্টে, স্বাভাবিক

পরীক্ষার ফাঁদ *(i)* সেগমেন্টেশন **বাহ্যিক**, পেজিং **অভ্যন্তরীণ** ফ্র্যাগমেন্টেশন দেয় — ঠিক বিপরীত রোগ। *(ii)* সেগমেন্ট নম্বর **পেজ নম্বর নয়**; সেগমেন্টের স্থির আকার নেই, তাই অ্যাড্রেস বিটে ভাগ করা যায় না। *(iii)* গোটা সেগমেন্ট শেয়ার করা স্বাভাবিক; **অর্ধেক** পেজ শেয়ার করা নয়।

পেজিংসহ সেগমেন্টেশন (segmented paging)

প্রতিটি সেগমেন্টকে **আবার** স্থির-আকারের পেজে ভাগ করা হয় এবং **প্রতিটি সেগমেন্টের নিজস্ব পেজ টেবিল থাকে**; সেগমেন্ট টেবিল এন্ট্রি এখন বেস+লিমিট জোড়ার বদলে **ওই সেগমেন্টের পেজ টেবিলের বেস অ্যাড্রেস** রাখে। লজিক্যাল অ্যাড্রেস *(s, p, d)* **তিন** ধাপে অনুবাদ হয় — সেগমেন্ট টেবিল → সেগমেন্ট *s*-এর পেজ টেবিল → তার ভেতরের পেজ *p*-এর ফ্রেম — ফলে `PA = f·P + d`। সেগমেন্ট টেবিল নিজে এত ছোট হতে পারে যে রেজিস্টারে থাকে, তাই কেবল দুটি মেমোরি সাফার খরচ হয়।

মনে রাখার কৌশল **"বেস যোগ করে, অফসেট বাহে, লিমিট ঠকায়।"** পেজিংয় ফ্রেম নম্বর পেজ নম্বরের **জায়গায়** বসে আর অফসেট নকল হয়; সেগমেন্টেশনে বেস অফসেটের সাথে **যোগ** হয়।

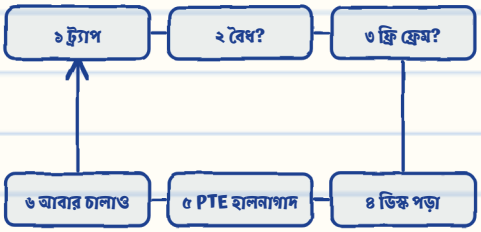
আসল মেশিন কী করে *x86* রিয়েল মোড হলো সরাসরি সেগমেন্টেশন; *x86* প্রোটেক্টেড মোড হলো **সেগমেন্টেড পেজিং** — একটি সিলেক্টর ডেসক্রিপ্টর বাহে, যা পেজ টেবিলের বেস দেয়, তারপর সাধারণ পেজিং চলে। *ARM64* ও লিনাক্স সেগমেন্টেশন বাদ দিয়েছে, কারণ ভিন্ন-দৈর্ঘ্য পার্টিশনের সমস্যাটা সমাধান করার মতো মূল্যবান নয়।

পরীক্ষার ফাঁদ **লিমিট** একটি **দৈর্ঘ্য**, শেষ অ্যাড্রেস নয়: বেস 44 ও লিমিট 24 হলে শেষ বৈধ অফসেট **23**, কারণ `d < লিমিট`। **সেগমেন্টেশন ফল্ট** আর **পেজ ফল্ট** ভিন্ন ধরনের ট্র্যাপ — প্রথমটি সেগমেন্টের লিমিট অতিক্রম, দ্বিতীয়টি মেমোরিতে না থাকা পেজ।

৬ ভার্চুয়াল মেমোরি, ডিমান্ড পেজিং ও পেজ ফল্ট

লিখিত প্রশ্ন ★

সংজ্ঞা **ভার্চুয়াল মেমোরি** *(Virtual memory)* এমন কৌশল যা প্রসেসকে শুধু অংশেই ফিজিক্যাল মেমোরিতে থেকে চালানোর সুযোগ দেয়: প্রোগ্রাম নিজস্ব **ভার্চুয়াল অ্যাড্রেস স্পেস** ব্যবহার করে, যা *RAM*-এর চেয়ে বড় হতে পারে, আর অব্যবহৃত অংশ *OS* ব্যাক স্টোরে রাখে। সাধারণ বাস্তবায়ন হলো **ডিমান্ড পেজিং** — **পেজ কেবল প্রথমবার রেফারেন্স হলেই আনা হয়**, যাকে **লেজি লোডিং**-ও বলা হয়। প্রতিটি *PTE*-তে একটি **ভ্যালিড–ইনভ্যালিড বিট** থাকে: ভ্যালিড = মেমোরিতে আছে, ইনভ্যালিড = নেই।



ছয় ধাপ, পরীক্ষক যে ক্রমে চাইবেন

৬ নম্বর ধাপ ঐচ্ছিক নয়

লুপটি বন্ধ হয় কারণ ফল্ট করানো ইনস্ট্রাকশনটি আবার শুরু থেকে চলে

পেজ ফল্ট সামলানোর ছয় ধাপ, ক্রমানুসারে

১. *MMU* দেখে **ভ্যালিড = 0** এবং **OS-এ ট্র্যাপ করে**; প্রসেস ব্লক হয়।

২. *OS* যাচাই করে রেফারেন্সটি **বৈধ কি না**। বৈধ নয় → **প্রসেস বন্ধ**।

৩. একটি **ফ্রি ফ্রেম** নেওয়া হয়; না থাকলে **পেজ রিপ্লেসমেন্ট অ্যালগরিদম** চালিয়ে ভিকটিম বাছে, আর তার **ডার্টি বিট** সেট থাকলে আগে ডিস্কে লেখে নেওয়া হয়।

৪. প্রয়োজনীয় পেজটি ওই ফ্রেমে আনার জন্য **ডিস্ক রিড** নির্ধারিত হয়; প্রসেস অপেক্ষা করে।

৫. পড়া শেষ হলে **পেজ টেবিল হালনাগাদ**: ফ্রেম নম্বর, **ভ্যালিড = 1**, ডার্টি = 0।

৬. ফল্ট করানো **ইনস্ট্রাকশনটি আবার চালানো** হয়।

সমাধানকৃত উদাহরণ ✍ কার্যকর অ্যাক্সেস সময়

প্রতি অ্যাক্সেসে $m = 100$ ns, প্রতি ফল্টে $f = 10$ ms, ফল্টের হার p। $EAT = (1 - p)\cdot m + p\cdot f$। (ক) $p = 10^{-6}$: $EAT \approx 100 + 10^{-6} \times 10^{7} = 110$ **ns**। (খ) ফল্টের বাড়তি একটি মেমোরি অ্যাক্সেসের নিচে রাখতে $p\cdot f < m$ লাগে, অর্থাৎ $p < 100\ ns \div 10^{7}\ ns = 10^{-5}$। (গ) $p = 10^{-3}$ হলে প্রায় **10 µs** — ১০০ গুণ ধীর। (ঘ) তাই ১০০ ০০০ রেফারেন্সে ১-এর বেশি ফল্ট হলে ঘাত কাটে — এর জন্যই ওয়ার্কিং সেট মডেল।

পরীক্ষার ফাঁদ *(i)* পেজ ফল্ট **কোনো এরর নয়** — এটিই পেজ আসার স্বাভাবিক পথ। *(ii)* ডার্টি বিট ডিস্ক রিডের **আগে** দেখা হয়, আর শুধু ডার্টি ভিকটিমই ফিরিয়ে লেখা হয়। *(iii)* "ইনভ্যালিড" মানে **মেমোরিতে নেই**, "ভাঙা" নয়। *(iv)* ভার্চুয়াল মেমোরি *RAM* **অনেক প্রসেসকে** ভাগ করে দিতে দেয়, কেবল একক প্রসেসকে *RAM* ছাড়ানো নয়।

মনে রাখার কৌশল **"ট্র্যাপ, টেস্ট, নাও, পড়ো, টেবিল লেখো, আবার।"** ছয় শব্দ, সবসময় এই ক্রমে। আর শেষ শব্দটিই ইঙ্গিত দেয়: ইনস্ট্রাকশনটি না চালালে প্রসেস চিরকাল মৃত থাকবে।

বাস্তবে রেফারেন্স কাউন্টিং ও গার্বেজ কালেকশন হলো একই ধারণা, ভিন্ন কাজে ব্যবহৃত: পরিচিত ভালো ইমেজ মেমোরিতে ম্যাপ করে ধীরে ধীরে ফল্ট করিয়ে আনা, আর অপরিবর্তিত অবস্থায় পেজ করে বের করা। লিনাক্স দ্বিতীয় ঘটনাকে **মাইনর ফল্ট** বলে — ডিস্ক *I/O* নেই, শুধু *PTE* পূরণ। ভার্চুয়াল মেমোরি সুবিধা হিসেবে দেওয়া হয় অনুবাদ, সুরক্ষা, শেয়ারিং ও প্রক্রিয়া বিচ্ছিন্নতা; পার্শ্বপ্রতিক্রিয়াগুলো ভেতরে থেকে দেখা যায় না।

৭ পেজ রিপ্লেসমেন্ট: চারটি অ্যালগরিদম ট্রেসটি অনুসরণ করো ★

সংজ্ঞা **পেজ রিপ্লেসমেন্ট** তখন চলে যখন প্রসেসের ফ্রেম দরকার কিন্তু ফ্রি ফ্রেম নেই: OS একটি মেমোরিতে থাকা পেজ বাছে, সেটি ডার্টি হলে ডিস্কে লেখে ফেরত আনে। পেজ ফল্ট রেট *PFR* = মোট পেজ ফল্ট ÷ মোট রেফারেন্স। **কোনো অ্যালগরিদমই পেজের প্রথম রেফারেন্স এড়ায় না**, আর **বেশি ফ্রেম মানেই কম ফল্ট নয়** — বেলাডির অ্যানোমালি।

অ্যালগরিদম	যে পেজটি বের করে	বাস্তবায়নযোগ্য?	মূল্যায়ন
FIFO	সবচেয়ে আগে লোড হওয়াটি, ব্যবহার মানে না দেখে	হ্যাঁ — একটি সারি	সহজ; বেলাডির অ্যানোমালি আছে
অপটিমাল (*MIN*)	যেটি **ভবিষ্যতে সবচেয়ে দেরিতে** লাগবে	না — ভবিষ্যৎ জানতে হয়	**মাপকাঠি, বাস্তব পছন্দ নয়**
LRU	যেটি **সবচেয়ে আগে** ব্যবহৃত হয়েছিল	ঠিকঠাক; বাস্তবে কাছাকাছি	ভালো; বেলাডির অ্যানোমালি হয় না
ক্লক ★	যেটি রেফারেন্স বিট = *0* থাকায় স্ক্যানে পাওয়া যায়	হ্যাঁ — সস্তা, অতিরিক্ত বিট লাগে না	*LRU*-র ধারণা, *LRU*-র দামে

সমাধানকৃত ট্রেস ✍ স্ট্রিং 1 2 3 4 1 2 5 1 2 3 4 5, তিন ফ্রেম, সব ফাঁকা — F = পেজ ফল্ট, "hit" = আগেই ফ্রেমে আছে

রেফ	পেজ	FIFO ফ্রেম	F/L	LRU ফ্রেম	F/L	অপটিমাল ফ্রেম	F/L
1	1	1 – –	F	1 – –	F	1 – –	F
2	2	1 2 –	F	1 2 –	F	1 2 –	F
3	3	1 2 3	F	1 2 3	F	1 2 3	F
4	4	4 2 3	F	2 3 4	F	1 2 4	F
5	1	4 1 3	F	3 4 1	F	1 2 4	hit
6	2	4 1 2	F	4 1 2	F	1 2 4	hit
7	5	5 1 2	F	1 2 5	F	1 2 5	F
8	1	5 1 2	hit	2 5 1	hit	1 2 5	hit
9	2	5 1 2	hit	5 1 2	hit	1 2 5	hit
10	3	5 2 3	F	1 2 3	F	3 2 5	F
11	4	5 3 4	F	2 3 4	F	3 4 5	F
12	5	5 3 4	hit	3 4 5	F	3 4 5	hit
ফল্ট			9		10		7

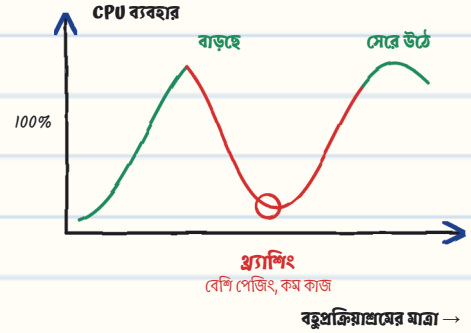
পরীক্ষার ফাঁদ বেলাডির অ্যানোমালি: **একই** স্ট্রিংয়ে **চার** ফ্রেমে *FIFO* দেয় ***10* টি ফল্ট — যা তিন ফ্রেমের চেয়ে বেশি**। তাই "বেশি ফ্রেম মানে কম ফল্ট" কথাটি *FIFO*-র ক্ষেত্রে **মিথ্যা**, *LRU* ও অপটিমালের ক্ষেত্রে সত্য। আর প্রতিটি আলাদা পেজের প্রথম রেফারেন্সে সবসময় ফল্ট হয়, তাই এই স্ট্রিংয়ে **কমপক্ষে ৫ টি ফল্ট** অনিবার্য।

মনে রাখার কৌশল *"MIN ছাদ, FIFO মেঝে।"* নিখুঁত *LRU*-র জন্য প্রতি ফ্রেমে টাইমস্ট্যাম্প বা স্ট্যাক লাগে, তাই আসল সিস্টেম **ক্লক** ব্যবহার করে, যা শুধু ফ্রেমগুলো ঘুরে রেফারেন্স বিট মুছে। *FIFO* ব্যবহার ভুলে যায়; *LRU* মনে রাখে; ক্লক সস্তায় মনে রাখে।

IT link লিনাক্স *LRU*-র কাছাকাছি করে **মাল্টি-জেন *LRU*** দিয়ে: পেজ টেবিল এন্ট্রিতে তরুণ/বৃত বিট থাকে, তাই কোনো স্ক্যান লাগে না। উইন্ডোজ ও ম্যাকওএস ক্লকেরই একটি রূপ ব্যবহার করে। পরীক্ষায় সবার নাম মোটামুটি *"LRU approximation"*।

৮ ওয়ার্কিং সেট, থ্র্যাশিং ও তার নিয়ন্ত্রণ শেষ টপিক ★

সংজ্ঞা প্রসেস i-এর সময় t-এ ওয়ার্কিং সেট হলো $WSS_i(t)$ = গত **Δ সময়ের** মধ্যে সে যে পেজগুলো রেফারেন্স করেছে (Δ ≈ কয়েক মিলিসেকেন্ড, যা তার স্বাভাবিক ওঠানামা ধরে রাখার মতো)। মোট চাহিদা $D(t) = \Sigma WSS_i(t)$। যদি $D(t)$ > **ফ্রি ফ্রেমের সংখ্যা**, পেজ বের করে কিছুক্ষণ পরেই আবার চাও হয় — এটিই **থ্র্যাশিং**।



ফ্রেম শেষ হওয়ার আগ পর্যন্ত আরেকটি প্রসেস যোগ করলে লাভ; তার পর ক্ষতি

থ্র্যাশিং থামানোর দুই পথ

১. ওয়ার্কিং-সেট নীতি। প্রতিটি প্রসেসকে ঠিক তার WSS-এর সমান ফ্রেম দিয়ে রাখো। ফল্ট হলে চাহিদা ফাঁকা ফ্রেমের চেয়ে বেশি হলে **যে প্রসেসের WSS কমছে তার পেজ বের করো**, অথবা পুরো একটি প্রসেস সাসপেন্ড করো। খালি ফ্রেম যায় যে প্রসেসের WSS বাড়ছে তার কাছে।

২. পেজ ফল্ট ফ্রিকোয়েন্সি (PFF) নিয়ন্ত্রণ। একটি সময়সীমায় প্রতিটি প্রসেসের PFR মাপো। PFR > **উপরের সীমা** হলে একটি ফ্রি ফ্রেম দাও; PFR < **নিচের সীমা** হলে ফেরত নাও। সরল, এবং Δ লাগে না।

লক্ষণ	অর্থ
PFR বাড়ছে	ওয়ার্কিং সেটের জন্য ফ্রেম কম
CPU ব্যবহার **কমছে**	ব্যবহৃত প্রসেস সময় নষ্ট করছে I/O-তে — থ্র্যাশিং
ডিস্ক লেখা বাড়ছে	ডার্টি পেজ বারবার ফেরত লেখা হচ্ছে

সমাধানকৃত উদাহরণ ✍ থ্র্যাশিং হবে কি? চারটি ফ্রেম ফাঁকা আছে। (ক) $WSS(P1) = 6$, $WSS(P2) = 2$, $WSS(P3) = 3$, তাই $D = 11 > 4$ → থ্র্যাশিং; ফ্রেম না হওয়া পর্যন্ত P3 সাসপেন্ড করো। (খ) এরপর P3-এর ওয়ার্কিং সেট 1 পেজে নেমে আসে; $D = 6 + 2 + 1 = 9$, এখনো > 4 → P2 সাসপেন্ড করো, তার WSS 0-তে নেমে এসেছে → $D = 7$। (গ) আরেকবার কমাও: $D = 6 \le$ ফ্রেমসংখ্যা → স্থিতিশীল, ব্যবহার আবার বাড়ে। (ঘ) PFF-এর চোখে: P1 প্রতি রেফারেন্সে 0.3 ফল্ট করছে, যা উপরের সীমার অনেক ওপরে — সে-ই সমস্যা। (ঙ) কেন থ্র্যাশিং ঘটে: P1 ও P2 একসাথে মিলে চায় 8 ফ্রেম, কিন্তু মাত্র 4 আছে, তাই দুজনেই একে অপরকে ধাক্কা দিচ্ছে — অথচ সিস্টেম **চলছে**, থ্র্যাশিং একটি অগ্রগতিহীন অবস্থা নয়, ধীর নিষ্ক্রগতি।

পরীক্ষার ফাঁদ (i) **থ্র্যাশিং ≠ ডেডলক**: সিস্টেম ব্যস্ত, আটকে নেই — পেজ একই একই আসা-যাওয়া করছে। (ii) থ্র্যাশিং **বেশি CPU ব্যবহার → কম** ব্যবহার; ডেডলকে কিছুই নড়ে না। (iii) প্রসেস যোগ করা **চাহিদা > ফ্রি ফ্রেম** হওয়া পর্যন্ত সবসময় সাহায্য করে, তারপর ক্ষতি করে।

মনে রাখার কৌশল "**চাহিদা বেশি, জায়গা কম = থ্র্যাশিং।**" ওয়ার্কিং সেটগুলো না গেলে OS হয় **প্রসেস সাসপেন্ড** করে, নয়তো **সবচেয়ে খিওর খাওয়াকে** ফ্রেম দেয়। আর সবসময় কাজ করা সমাধানটি একটু সাধারণ: কম প্রসেস ঢোকাও।

থ্র্যাশিং চেনার তিন লক্ষণ

- PFR ক্রমাগত বাড়ছে
- CPU ব্যবহার **কমছে**
- ডিস্ক লেখা-পড়া বাড়ছে
- একই প্রসেস বারবার ফল্ট করছে

রাতের পাতা

তথ্য-পত্র

বাইন্ডিং: কম্পাইল / লোড / এক্সিকিউশন
লজিক্যাল = 0 থেকে, **রিলেটিভ**
বেস + লিমিট ⇒ কনটিগুয়াস
বাহ্যিক = ব্লকের মাঝে
পেজ = ফ্রেম আকার, **স্থির**
$PA = f \cdot P + d$
32-bit + 4 KB ⇒ 2^2 এন্ট্রি
টিএলবি হিট = 1, মিস = 2
ডার্টি = বদলেছে · রেফারেন্স = দেখা
$d \geq Limit[s]$ ⇒ ফল্ট
সেগমেন্ট **বাড়ে** ⇒ বাহ্যিক ফ্র্যাগ
ডিমান্ড পেজিং = **লেজি**
$EAT = (1 - p)m + p \cdot f$
LRU ও MIN-এ কখনো না
WSS = গত Δ-এর পেজ
সমাধান: সাসপেন্ড, বা PFF

এক্সিকিউশন = **রিলোকেশন**
ফিজিক্যাল = RAM ইউনিটে
অভ্যন্তরীণ = ব্লকের ভেতরে
কম্প্যাকশন কেবল বাহ্যিক ঠিকায়
$p = LA$ **div** $P \cdot d = LA$ **mod** P
$P = 2^n$ ⇒ অফসেট = **n বিট**
এক স্তরের টেবিল = **4 MB**
PTE: ভ্যালিড, প্রোটেকশন, ডার্টি, রেফারেন্স
সেগমেন্ট: $PA = Base[s] + d$
সেগ + পেজিং ⇒ (s, p, d)
VM: প্রসেসের অংশই RAM-এ
ফল্ট: ট্র্যাপ→টেস্ট→নাও→পড়ো→PTE→আবার
FIFO-তে **বেলাডি** হতে পারে
ক্লক = সস্তা LRU
$D(t) >$ ফ্রি ফ্রেম ⇒ **থ্র্যাশিং**
PFR = ফল্ট ÷ রেফারেন্স

➜ MCQ-র জন্য সেরা ১২টি এক-লাইনের কথা

1. ফিজিক্যাল মেমোরি = **ফ্রেম**, লজিক্যাল = **পেজ**
2. অফসেট = LA **mod** P, **হুবহু নকল** হয়
3. $P = 2^n$ ⇒ অফসেট ঠিক **n বিট**
4. P 1 KB, পেজ 2 → ফ্রেম 5, LA 3000 ⇒ **6072**
5. টিএলবি হিট = **এক** মেমোরি অ্যাক্সেস
6. টিএলবি মিস = **দুই** মেমোরি অ্যাক্সেস
7. 32-bit + 4 KB পেজ ⇒ 2^2 এন্ট্রি
8. 4 বাইটের PTE ⇒ **4 MB** এক স্তরের টেবিল
9. সেগমেন্ট ফল্ট হয় যখন $d \geq Limit[s]$
10. পেজিং ⇒ **অভ্যন্তরীণ** ফ্র্যাগমেন্টেশন
11. VM ⇒ **ডিমান্ড পেজিং**, ভ্যালিড-ইনভ্যালিড বিট
12. থ্র্যাশিং ⇒ **কম প্রসেস**

সবচেয়ে বড় ফাঁদগুলো অফসেট বেসের সাথে **এবং** ফ্রেমের সাথে দুইবার যোগ · পেজিং সব ফ্র্যাগমেন্টেশন ঠিকায় বলা · FIFO বেলাডি থেকে অব্যাহতিপ্রাপ্ত · ডার্টি বিটকে রেফারেন্স বিটের সাথে গুলিয়ে ফেলা · সেগমেন্টেশনের ব্লক স্থির-আকারের বলা · পেজ ফল্টকে এরর ভাবা · "বেশি ফ্রেম মানেই কম ফল্ট" · ভার্চুয়াল মেমোরি **একক** প্রসেসকে RAM ছাড়তে দেয় বলা।

সব কৌশল একসাথে "কম্পাইল = জমে গেল, লোড = বাঁধা, রান = চলমান" · "প্রোগ্রাম লজিক্যাল দেখে, শুধু চিপ ফিজিক্যাল" · "ভেতরে = অভ্যন্তরীণ, বাইরে = বাহ্যিক" · "পেজ ঢোকে, ফ্রেম বেরোয়" · "হিট একবার, মিস দুইবার" · "D = Dirty, R = Referenced" · "বেস যোগ করে, অফসেট বাছে, লিমিট ঠকায়" · "ট্র্যাপ, টেস্ট, নাও, পড়ো, টেবিল লেখো, আবার" · "MIN ছাদ, FIFO মেঝে" · "চাহিদা বেশি, জায়গা কম = থ্র্যাশিং"।

সোয়াপিং
পুরো প্রসেস সরায়
OS-এর নিজস্ব সিদ্ধান্তে

পেজিং
একটি স্থির-আকারের পেজ
প্রোগ্রামের রেফারেন্সে

সেগমেন্টেশন
একটি লজিক্যাল সেগমেন্ট
ভিন্ন দৈর্ঘ্যের

"কী সরে, কত বড়, আর কে ঠিক করল"

তিন পদ্ধতি আলাদা করার তিনটি শব্দ – এই প্রশ্নটি বারবার আসে

✓ অনুশীলনী MCQ (পরীক্ষার ধরনে, ১০ নম্বর)

আগে চেষ্টা করো, তারপর দেখো

১. পেজিং বাহ্যিক ফ্র্যাগমেন্টেশন ঠিকায়, তবে কী থেকে পীড়ায়:

ক) স্টারভেশন খ) রেস কন্ডিশন
গ) কনভয় ইফেক্ট ঘ) অভ্যন্তরীণ ফ্র্যাগমেন্টেশন

২. চালানোর চেয়ে পেজ বদলাতেই বেশি সময় কাটে, মানে:

ক) থ্র্যাশিং খ) স্টারভেশন
গ) ডেডলক ঘ) ফ্র্যাগমেন্টেশন

৩. পেজ 1 KB, পেজ 2 → ফ্রেম 5, লজিক্যাল 3000। ফিজিক্যাল:

ক) 5072 খ) 3952
গ) 6072 ঘ) 7096

৪. 32-bit অ্যাড্রেস, 4 KB পেজ। একটি টেবিলে এন্ট্রি:

ক) 2^{12} খ) 2^{20}
গ) 2^{32} ঘ) 2^{10}

৫. PTE-র **ডার্টি বিট** কখন সেট হয়:

ক) ফ্রেমে লোড হলে খ) প্রথম রেফারেন্সে
গ) পরিবর্তিত হলে ঘ) সোয়াপ আউটে

৬. টিএলবি 20 ns, মেমোরি 100 ns, হিট 80 %। অ্যাক্সেস সময়:

ক) 120 ns খ) 140 ns
গ) 150 ns ঘ) 220 ns

৭. সেগমেন্টেশনে অফসেট **লিমিটের সমান বা বেশি** হলে কী হয়:

ক) টিএলবি মিস খ) পেজ ফল্ট
গ) কম্প্যাকশন ঘ) সেগমেন্টেশন ফল্ট

৮. 4 KB পেজ সাইজে অফসেট ফিল্ডে কত বিট লাগে:

ক) 4 বিট খ) 12 বিট
গ) 20 বিট ঘ) 32 বিট

৯. ডিমান্ড পেজিং একটি পেজ কখন আনে:

ক) লোডের সময় খ) আগেভাগে
গ) কখনো না ঘ) প্রথম রেফারেন্সে

১০. পেজিংসহ সেগমেন্টেশন মেমোরিকে কীভাবে ঠিকানা দেয়:

ক) (p, d) খ) (s, d)
গ) (s, p, d) ঘ) একটি সরল রেখা অ্যাড্রেস

উত্তরমালা → ১-ঘ · ২-ক · ৩-গ · ৪-খ · ৫-গ · ৬-খ · ৭-ঘ · ৮-খ · ৯-ঘ · ১০-গ

লিখিত প্রশ্ন ভার্চুয়াল মেমোরি কী? ডিমান্ড পেজিং ব্যাখ্যা করো এবং একটি পেজ ফল্ট সামলাতে অপারেটিং সিস্টেম কী কী ধাপ অনুসরণ করে তা লেখো। ১০ নম্বর

১ **ভার্চুয়াল মেমোরি** — এমন কৌশল যা প্রসেসকে তখনই চালানোর সুযোগ দেয় যখন তার কেবল একটি অংশই ফিজিক্যাল মেমোরিতে আছে; প্রোগ্রাম নিজস্ব **ভার্চুয়াল অ্যাড্রেস স্পেস** ব্যবহার করে, যা RAM-এর চেয়ে বড় হতে পারে, আর অব্যবহৃত পেজগুলো OS ব্যাক স্টোরে রাখে। ৩

২ **ডিমান্ড পেজিং** — পেজ **চাহিদামতো** আনা হয়, কেবল প্রথমবার রেফারেন্স হলেই (lazy loading)। প্রতিটি PTE-তে একটি **ভ্যালিড-ইনভ্যালিড বিট** থাকে: ভ্যালিড = পেজ মেমোরিতে আছে, ইনভ্যালিড = নেই। ৩

৩ **ছয় ধাপ, ক্রমানুসারে** — (১) MMU ভ্যালিড = 0 পেয়ে **OS-এ ট্র্যাপ করে**, প্রসেস ব্লক হয়; (২) OS যাচাই করে রেফারেন্সটি **বৈধ কি না**, নয়তে প্রসেস বন্ধ করে; (৩) একটি **ফ্রি ফ্রেম** নেয়, না থাকলে **পেজ রিপ্লেসমেন্ট অ্যালগরিদম** চালিয়ে ভিকটিম বাছে এবং **তার ডার্টি বিট সেট থাকলে আগে ডিস্কে লেখে নেয়**; (৪) প্রয়োজনীয় পেজটি ওই ফ্রেমে আনতে **ডিস্ক রিড নির্ধারণ** করে; (৫) পড়া শেষ হলে **পেজ টেবিল হালনাগাদ** করে — ফ্রেম নম্বর, ভ্যালিড = 1, ডার্টি = 0; (৬) ফল্ট করানো **ইনস্ট্রাকশনটি আবার চালায়**। ৪

→ পরীক্ষার আগে ✓

- ✓ অ্যাড্রেসকে পেজ নম্বর ও অফসেটে ভাগ করতে পারি
- ✓ PTE থেকে ফিজিক্যাল অ্যাড্রেস বের করতে পারি
- ✓ FIFO ও LRU-তে রেফারেন্স স্ট্রিং ট্রেস করতে পারি
- ✓ পেজ ফল্টের ছয় ধাপ ক্রমানুসারে বলতে পারি